

LABORATORY OBSERVATION

Course	Full Stack Web Development
Course Code	23CM4121
Experiment No.	5
Student Name	Kavya G.
Roll No.	A24126552082
Date	21-09-2026

1. TITLE

Node.js File Management Using the File System Module

2. AIM

To create a Node.js application that accepts a filename and content from the user and performs file creation/writing, reading, appending additional content, and displaying the final file contents using the Node.js File System module.

3. OBJECTIVE

- To understand the basics of Node.js server-side scripting.
 - To understand the Node.js File System (fs) module.
 - To accept filename and content from the user.
 - To create and write content to a file.
 - To read the contents of an existing file.
 - To append additional content to a file.
 - To display the final contents of the file.
 - To use the readline module for command-line input.
-

4. THEORY

Node.js is a JavaScript runtime environment that allows JavaScript programs to run outside the browser. It can be used to develop server-side applications and command-line scripts.

The File System (fs) module in Node.js provides functions for working with files and directories. In this application, the fs module is used to create/write a file, read its contents, append additional content, and read the final contents.

The readline module is used to receive input from the user through the command line. A readline interface connects process.stdin for input and process.stdout for output.

The main file operations performed are:

- fs.writeFileSync() – creates a file or overwrites an existing file with the given content.
- fs.readFileSync() – reads the contents of the file.
- fs.appendFileSync() – adds additional content to the end of the file.

- `readline.question()` – accepts input from the user.
- `rl.close()` – closes the readline interface after the operations are completed.

Basic flow: User Input → Create/Write File → Read File → Append Content → Read Final File → Display Output

5. ALGORITHM / PROCEDURE

1. Import the File System module using `require('fs')`.
 2. Import the readline module using `require('readline')`.
 3. Create a readline interface using `process.stdin` and `process.stdout`.
 4. Ask the user to enter a filename.
 5. Ask the user to enter the initial content.
 6. Create the file and write the initial content using `fs.writeFileSync()`.
 7. Display a success message after the file is created and written.
 8. Read the original file contents using `fs.readFileSync()` with UTF-8 encoding.
 9. Display the original contents.
 10. Ask the user to enter additional content.
 11. Append the additional content using `fs.appendFileSync()`.
 12. Display a success message after appending.
 13. Read the final contents using `fs.readFileSync()`.
 14. Display the final contents.
 15. Close the readline interface using `rl.close()`.
-

6. PROGRAM

```
const fs = require('fs');
const readline = require('readline');

// Create readline interface
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

// Get filename from user
rl.question('Enter filename: ', (filename) => {

  // Get initial content
  rl.question('Enter content: ', (content) => {
```

```

// Create and write file
fs.writeFileSync(filename, content);
console.log('File created and written successfully!');

// Read file contents
let data = fs.readFileSync(filename, 'utf8');
console.log('Original content:', data);

// Get additional content
rl.question('Enter additional content: ', (additional) => {

    // Append content
    fs.appendFileSync(filename, '\n' + additional);
    console.log('Content appended successfully!');

    // Display final contents
    let finalData = fs.readFileSync(filename, 'utf8');
    console.log('Final contents:\n' + finalData);

    rl.close();
});
});
});

```

7. CODE EXPLANATION

Code / Function	Explanation
const fs = require('fs');	Imports the Node.js File System module.
const readline = require('readline');	Imports the readline module for command-line input.
readline.createInterface()	Creates an interface for receiving user input and displaying output.
process.stdin	Represents standard input from the user.
process.stdout	Represents standard output.
rl.question()	Asks the user for input and executes a callback with the entered value.
fs.writeFileSync()	Creates/writes the file with the initial content.
fs.readFileSync()	Reads the contents of the file.
'utf8'	Specifies UTF-8 text encoding while reading.
fs.appendFileSync()	Adds additional content to the end of the file.
console.log()	Displays messages and file contents in the terminal.
rl.close()	Closes the readline interface after completing the operations.

8. TEST CASES AND EXECUTION DATA

Test Case	Filename	Initial Content	Additional Content	Expected Result
-----------	----------	-----------------	--------------------	-----------------

TC-01	notes.txt	Hello Node.js	Welcome to File System	Both lines displayed.
TC-02	sample.txt	First content	Second content	Initial and appended content displayed.
TC-03	data.txt	Node.js File Handling	Append operation successful	Final contents displayed successfully.

9. OUTPUT

TC-01

```

Enter filename: notes.txt
Enter content: Hello Node.js
File created and written successfully!
Original content: Hello Node.js
Enter additional content: Welcome to File System
Content appended successfully!
Final contents:
Hello Node.js
Welcome to File System

```

TC-02

```

Enter filename: sample.txt
Enter content: First content
File created and written successfully!
Original content: First content
Enter additional content: second content
Content appended successfully!
Final contents:
First content
second content

```

TC-03

```

Enter filename: data.txt
Enter content: Node.js File Handling
File created and written successfully!
Original content: Node.js File Handling
Enter additional content: Append operation successful
Content appended successfully!
Final contents:
Node.js File Handling
Append operation successful

```

10. RESULT

Thus, a Node.js File Management application was successfully developed using the File System module. The application accepts a filename and content from the user, creates and writes the file, reads the original contents, appends additional content, and displays the final contents successfully.